


PRESIDENCY UNIVERSITY

# CSE321: Microprocessors Based System Design

## Lecture 01: Microprocessor Fundamentals & System Components

Microprocessor, Microcomputer, CPU, Registers, Buses, Memory and I/O

**Afzal Hossan**

**Lecturer**

Department of Computer Science and Engineering

**Presidency University**

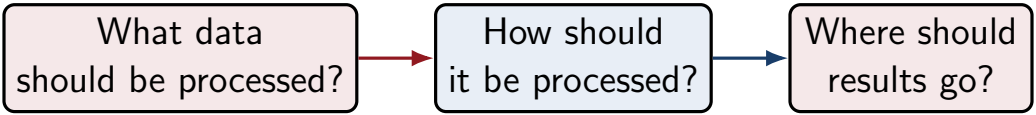
# Lecture 01: Learning Objectives

By the end of this lecture, students should be able to:

1. Explain the role of a **microprocessor** in a computer system.
2. Distinguish between a **microprocessor, microcontroller, and microcomputer**.
3. Identify the major components of a **microprocessor-based system**.
4. Explain the functions of the **ALU, Control Unit, and registers**.
5. Describe the purpose of **address, data, and control buses**.
6. Explain how the processor communicates with **memory and I/O devices**.
7. Describe the basic idea of the **fetch–decode–execute cycle**.
8. Explain why counters and timing/control signals are important in processor operation.

# Where Does This Course Begin?

A computer system must answer three fundamental questions:




Input $\rightarrow$ Processing $\rightarrow$ Output

The **microprocessor** provides the central processing capability that coordinates these activities.

# What is a Microprocessor?

## Definition

A **microprocessor** is a programmable digital processor in which the main processing functions of a CPU are implemented on a single integrated circuit (IC).

A microprocessor:

- accepts instructions,
- processes data according to those instructions,
- performs arithmetic and logical operations,
- controls the movement of information, and
- communicates with memory and peripheral devices.

## Key Idea

A microprocessor is primarily the **processing engine** of a microprocessor-based system. Memory and many peripheral functions are typically provided by other components.

# Microprocessor vs. Microcontroller

| **Feature** | Microprocessor | Microcontroller |
|---|---|---|
| **Main role** | General-purpose processing | Dedicated embedded control |
| **CPU** | Included | Included |
| **Memory** | Often external | Usually integrated |
| **I/O peripherals** | Often external | Usually integrated |
| **Typical use** | Computers, complex systems | Appliances, sensors, controllers |

**Important**

The distinction is about **system integration and intended use**, not simply about which device is "more powerful."

# What is a Microcomputer?

**Definition**

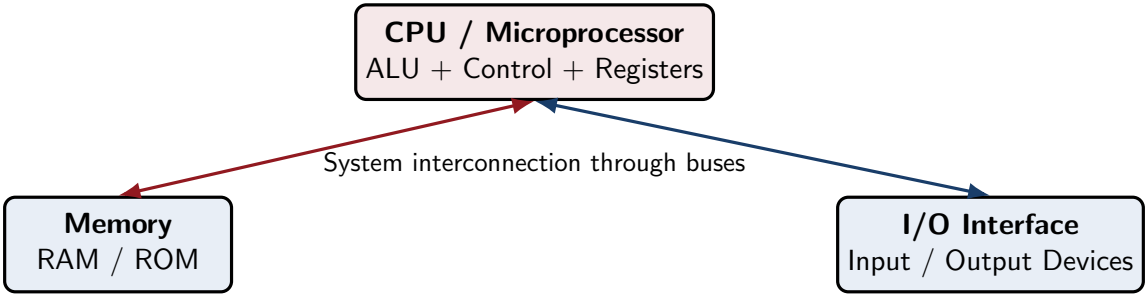
A **microcomputer** is a computer system built around a microprocessor as its CPU.

A basic microcomputer contains:

- a microprocessor (CPU),
- memory,
- input/output interfaces,
- buses for communication,
- clock and timing circuitry, and
- peripheral devices.

$$\text{Microprocessor} + \text{Memory} + \text{I/O} + \text{Buses} \Rightarrow \text{Microcomputer System}$$

# Basic Microprocessor-Based System




CPU $\longleftrightarrow$ Memory $\longleftrightarrow$ I/O

# Major Components of the CPU

A processor can be viewed conceptually as three major parts:




- **ALU:** performs arithmetic and logical operations.
- **Control Unit:** directs the sequence of operations.
- **Registers:** hold operands, addresses, instructions, and intermediate results.

# Arithmetic Logic Unit (ALU)

## Role of the ALU

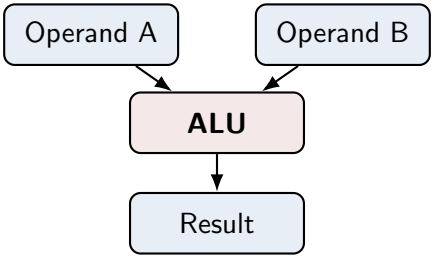
The **Arithmetic Logic Unit** is the part of the CPU responsible for arithmetic and logical processing.

**Arithmetic operations**

- Addition / Subtraction
- Increment / Decrement

**Logical operations**

- AND / OR
- XOR



# Control Unit

The **Control Unit (CU)** coordinates processor operations.
It determines:

- which operation should occur,
- when an operation should occur,
- which register should provide data,
- where data should be transferred, and
- which control signals should be activated.

Instruction → Control Unit → Control Signals

**Think of the CU as the coordinator**

The ALU performs an operation, but the Control Unit determines **what happens, when it happens, and where the results go**.

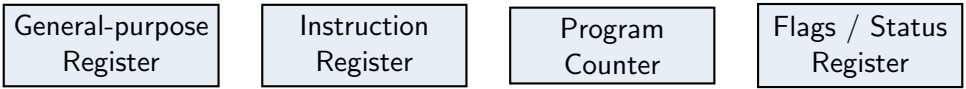
# Registers: The CPU's Fast Working Storage

**Definition**

A **register** is a small, high-speed storage location inside the processor used to hold information temporarily.

Registers may hold:

- operands,
- intermediate results,
- memory addresses,
- the current instruction,
- the next instruction address, and
- status information.



# Types of Registers

Different registers serve different purposes.

| **Register** | Purpose |
|---|---|
| **General-purpose register** | Holds data and intermediate results |
| **Accumulator** | Holds arithmetic/logic results |
| **Program Counter (PC)** | Holds address of next instruction |
| **Instruction Register (IR)** | Holds current instruction |
| **Memory Address Register (MAR)** | Holds memory address |
| **Memory Data Register (MDR)** | Holds data transferred to/from memory |
| **Status/Flag Register** | Holds processor status information |

**Important**

The exact set of registers depends on the processor architecture. We will study the actual 8086 registers in a later class.

# What is a Bus?

**Definition**

A **bus** is a group of electrical signal lines used to transfer information between components of a computer system.

A microprocessor-based system mainly uses:

1. **Address Bus**
2. **Data Bus**
3. **Control Bus**

$$\text{Address} + \text{Data} + \text{Control} \Rightarrow \text{System Communication}$$

# Address Bus

The **address bus** identifies the location involved in a memory or I/O operation.




If a processor has an $n$-bit address bus, it can represent:

$$\boxed{2^n \text{ distinct addresses}}$$

For example, a 20-bit address bus can represent:

$$2^{20} = 1,048,576$$

distinct addresses.

# Data Bus

The **data bus** carries the actual information being transferred.




Examples of data transferred over the data bus:

- instruction bytes,
- operands,
- results,
- input data from peripherals.

**Important**

Unlike the address bus, the data bus normally carries information in **both directions**.

# Control Bus

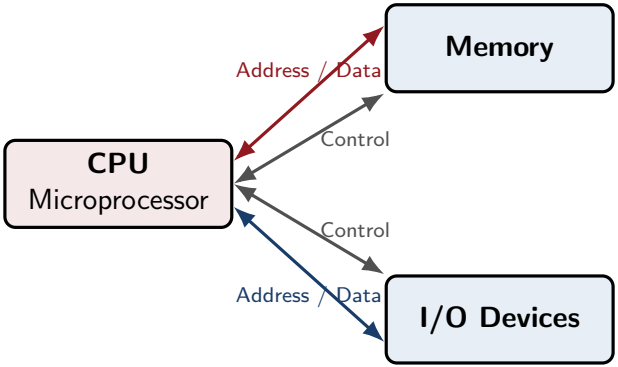
The **control bus** carries signals that coordinate system operations.
Typical control signals include:

- Read
- Write
- Interrupt request
- Reset
- Clock-related control
- Bus request / bus acknowledge




| Address: Where? | Data: What? | Control: How/When? |
|---|---|---|

# The Three Buses Together




## Core Concept

The CPU does not work in isolation. It continuously communicates with memory and I/O through the system buses.

# Memory: Where Programs and Data Reside

Memory provides storage locations that the processor can access.

## RAM

- Read/write memory
- Usually volatile
- Stores active programs and data

## ROM / Non-volatile Memory

- Retains information without power
- Stores firmware or fixed program information

$$\text{CPU} \xrightarrow{\text{address} + \text{control}} \text{Memory} \xleftrightarrow{\text{data}} \text{CPU}$$

# Memory Addressing: A Simple Example

Suppose a system has a memory space containing locations:

$$0000,\ 0001,\ 0002,\ \ldots,\ \mathit{FFFF}$$

Each address identifies a particular memory location.

If the CPU wants to read data from address 2050:

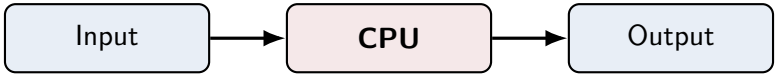
1. The CPU places 2050 on the **address bus**.
2. The CPU activates the appropriate **control signal**.
3. Memory places the requested value on the **data bus**.
4. The CPU receives the data.

Address → Select Location → Read/Write → Transfer Data

# Input/Output (I/O)

A processor needs to communicate with the outside world.
Examples of I/O devices:

- keyboard,
- display,
- printer,
- sensor,
- storage device,
- communication interface.




I/O allows the processor to interact with the physical system.

# Memory-Mapped I/O vs. Isolated I/O

Two common ways to organize I/O addressing are:

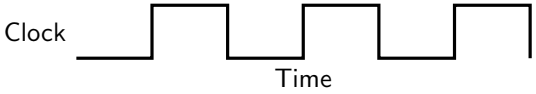
| **Feature** | Memory-Mapped I/O | Isolated / Port-Mapped I/O |
|---|---|---|
| **Address space** | I/O shares memory address space | Separate I/O address space |
| **Access mechanism** | Memory instructions may access devices | Dedicated I/O instructions may be used |
| **Design idea** | Treat device registers like memory locations | Treat I/O ports separately |

**8086 Connection**

The Intel 8086 supports a separate I/O address space, which we will study when we discuss its memory and bus organization.

# Clock and Timing

A processor needs timing so that operations occur in a coordinated order.




The clock provides a sequence of timing events used to coordinate:

- instruction fetching,
- instruction decoding,
- data transfers,
- ALU operations, and
- memory/I/O operations.

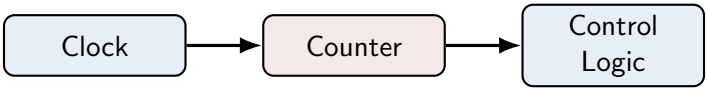
**Key Idea**

Processor operations are not random events; they occur according to a controlled sequence of timing signals.

# Counters and Timing Control

A **counter** is a sequential circuit that changes its state according to clock pulses.
Counters can be used to:

- count timing steps,
- sequence control operations,
- generate addresses,
- track instruction or machine cycles, and
- divide clock frequencies.



# Instruction Execution: The Basic Idea

A processor repeatedly performs a cycle:

$$\boxed{\text{Fetch} \rightarrow \text{Decode} \rightarrow \text{Execute}}$$

**1. Fetch**

The processor obtains the next instruction from memory.

**2. Decode**

The Control Unit interprets what operation the instruction requests.

**3. Execute**

The processor performs the operation and updates the required registers, memory locations, or I/O devices.

# A Simple Example: ADD Two Values

Suppose:

$$A = 5, \qquad B = 3$$

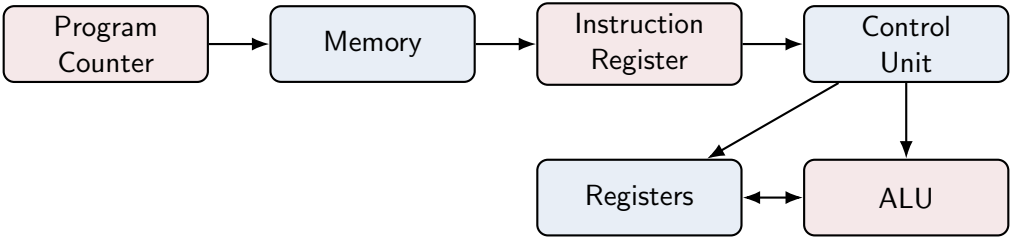
and the program requests:

$$\boxed{\text{ADD } A, B}$$

Conceptually:

1. **Fetch:** obtain the ADD instruction.
2. **Decode:** determine that an addition is required.
3. **Read operands:** obtain 5 and 3.
4. **Execute:** ALU computes
$$5 + 3 = 8$$
5. **Store result:** place 8 in the destination register or memory location.

# Putting the Components Together




Instructions control data movement and processing.

# Why Study SAP-1 Before the 8086?

The next lectures introduce the **SAP-1** architecture.
SAP-1 is useful as a conceptual bridge because it makes processor operation easier to visualize:

Registers → Bus → Memory → Control → Instruction Execution

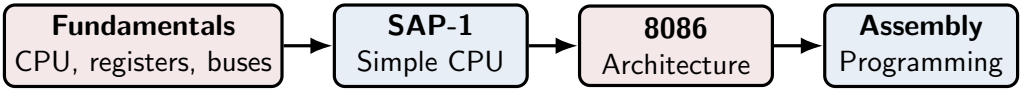
Then we move toward the much richer:

**Intel 8086**

## Course Roadmap

The goal is not to memorize isolated components. The goal is to understand how the components cooperate to execute instructions.

# From Fundamentals to 8086




Later:

8086 → Memory / I/O Interfacing → Peripheral Devices → ADC

# Quick Check #1

Which component primarily performs arithmetic and logical operations?

1. Control Unit
2. ALU
3. Program Counter
4. Address Bus

**Answer**

ALU

# Quick Check #2

A processor has a 16-bit address bus.

How many distinct addresses can it represent?

$$2^{16} = 65,536$$

Therefore:

$$\boxed{65,536 \text{ distinct addresses}}$$

**General rule: an $n$-bit address bus provides $2^n$ addresses.**

# Quick Check #3

Match each bus with its primary purpose:

| **Bus** | Primary Function |
|---|---|
| **Address Bus** | ? |
| **Data Bus** | ? |
| **Control Bus** | ? |

**Think**

Address = Where  Data = What  Control = How/When

# Lecture Summary

Today we established the foundation of a microprocessor-based system.

1. A **microprocessor** provides the central processing function.
2. A **microcomputer** combines the processor with memory, I/O, and supporting components.
3. The CPU contains the **ALU, Control Unit, and registers**.
4. **Address, data, and control buses** connect the processor with the rest of the system.
5. **Memory** stores instructions and data.
6. **I/O** connects the processor to external devices.
7. The processor operates through a **fetch–decode–execute** cycle.
8. Timing, counters, and control signals coordinate operations.

# Remember These Four Ideas

$$\boxed{\text{CPU} = \text{ALU} + \text{Control} + \text{Registers}}$$

$$\boxed{\text{System Communication} = \text{Address} + \text{Data} + \text{Control}}$$

$$\boxed{\text{Instruction Cycle} = \text{Fetch} \rightarrow \text{Decode} \rightarrow \text{Execute}}$$

# Preparation for Lecture 02

Before the next lecture, review:

- What is a microprocessor?
- What are the three major parts of a CPU?
- What is the role of each system bus?
- What is the difference between memory and I/O?
- What happens during fetch, decode, and execute?
- Why are registers faster and more directly accessible than main memory?

**Next Lecture**

**SAP-1 Architecture**

Registers, ALU, Control Unit, Bus, Memory and Instruction Cycle